

Python 两周入门手册

一、教学目标

本教程旨在帮助零基础学习者在两周时间内掌握 Python 的基本语法和核心编程技能，能够使用常见的数据结构编写程序，理解并能够编写简单的函数，初步掌握类与对象的使用。

最终，同学们能够通过综合大作业将所学知识应用于实践，完成一个小型系统。

【注】本教程的功能仅在于入门学习，适合初学者快速上手 Python 编程。不能够涵盖 Python 的所有高级特性和应用场景，**不能**替代系统的编程课程。

推荐学习资料

更系统的学习资料请参考以下资源：

在线教程：

- [python官方文档教程](#)：官方教程，**强烈推荐**！
- [菜鸟教程 Python3 教程](#)：提供大量实例与清晰解释，从基础语法到高级特性，循序渐进引导学习，非常适合初学者。
- [廖雪峰的 Python 入门教程](#)：讲解详细全面，涵盖 Python 编程的各个方面，从基础到进阶，能帮助学习者系统构建知识体系。

书籍：

- 《Python 编程：从入门到实践（第 2 版）》：全面介绍 Python，前半部分讲解编程基础概念，后半部分通过开发项目将理论付诸实践，适合零基础读者开启 Python 学习之路。
- 《Python 编程快速上手：让繁琐工作自动化》：聚焦实践应用，不仅介绍 Python 基础知识，还通过实际项目教会读者如何运用 Python 解决日常工作中的繁琐任务，让编程变得实用有趣。

视频课程：

- B 站“Python 从入门到实践”系列视频：众多 UP 主制作的优质教程，以生动形象的方式讲解 Python 知识，适合喜欢通过视频学习的人群，零基础也能轻松跟上节奏。例如[黑马程序员的 python 入门教程](#)（BV号：BV1qW4y1a7fU，**强烈推荐**）
- 网易云课堂、腾讯课堂等平台的 Python 课程：平台上有许多专业讲师录制的课程，从基础到进阶，不同难度层次均有覆盖，且部分课程提供答疑服务，方便学习者解决学习过程中遇到的问题。

以上资源均适合零基础学习者，能够帮助你系统掌握 Python 编程的核心知识与技能。建议结合多种形式的学习资料，选一个你自己**能接受学进去**的作为主要学习材料，其他的作为辅助，理论与实践相结合，提升编程能力。

二、教程安排

教程共 14 天，分为两周进行。第一周重点掌握基础语法与数据结构，第二周重点学习函数与类，并完成综合项目。

本教程的每天的学习内容包括：

- **知识点**：当天需要掌握的核心概念与语法。
- **学习资料**：推荐的学习资源链接，基于**python官方文档**和**菜鸟教程**等资料，便于查阅。可以根据个人喜好选择其他资料。
- **应知应会 Checklist**：当天学习结束后需要掌握的技能点。
- **小任务**：通过编写代码完成的小练习，巩固当天所学内容。

【注】本教程提供的小任务和大作业均为参考示例，学习者可根据自身兴趣和需求进行调整和扩展。**不强制要求完成**，完成情况**不计入最终考核评分**。

三、预期成果

- 同学们能独立编写 Python 程序解决简单问题
- 能熟练使用基本语法、函数、数据结构
- 理解类与对象的概念，并能编写基础类
- 完成一个完整的“学生成绩管理系统”大作业

四、每日学习内容

项目结构准备

在开始正式学习之前，我们需要先准备好项目的基本目录结构，方便每天的代码和总结文档进行管理。

【注】本项目结构仅为建议，学习者可根据个人习惯进行调整，但建议保持清晰有序，便于后续复习和查阅。

操作步骤

1. 打开你的工作目录（例如 `D:\my-first-project` 或 `~/my-first-project`）。
2. 在其中新建一个文件夹：

```
my-first-project/  
└─ python学习/
```

3. 在 `python学习` 文件夹下，建立 Day 0 ~ Day 14 共 **15 个子文件夹**，用于存放每天的代码和练习：

```
python学习/  
├─ Day0/  
├─ Day1/  
├─ ...  
└─ Day14/
```

文件组织规范

- 每个 `DayX/` 文件夹下，至少包含：
 - 当天的小任务代码文件（例如 `task.py`）。
 - 如果有总结或笔记，推荐使用 **Markdown 文件**（例如 `notes.md`）来记录当天的学习要点。

例如，`Day1/` 文件夹可能长这样：

```
Day1/  
├─ task.py      # 小任务代码  
└─ notes.md     # 知识点总结
```

提交至 git 与 GitHub

1. 将 `my-first-project` 文件夹初始化为一个 Git 仓库：

```
cd my-first-project  
git init
```

2. 每天学习结束后，将当天的代码和笔记提交到 Git 仓库：

```
git add .  
git commit -m "完成 DayX 学习任务"
```

3. 如果有远程仓库（例如 GitHub），可以将本地仓库与远程仓库关联，并推送代码：

```
git remote add origin <你的远程仓库地址>
git push -u origin main
```

【注】推荐将每日的进度提交到 Git 仓库和 GitHub，养成良好的版本管理习惯，便于后续复习和查阅。

推荐做法

- **代码文件：**将每天的小任务实现写在 `task.py` 或拆分成多个小文件。
- **笔记文件：**使用 Markdown 简洁总结：
 - 当天学习的语法/概念
 - 小任务的实现思路
 - 遇到的问题与解决办法

这样到第 14 天结束时，你将拥有一个完整的 **学习代码库 + 学习日志**，方便复习与回顾。

Day 0: 环境配置

0.1 基本知识点

- 安装 Python 3.x
- 配置 IDE（PyCharm / VSCode）
- 使用终端运行 Python 代码
- 编写并运行第一个 Python 程序

0.2 学习资料

- **环境搭建：**
 - [Python 官方下载](#)
 - [VSCode 官方教程](#)
 - [PyCharm 官方教程](#)
 - [菜鸟教程 - Python3 环境搭建](#)
- Python 官方教程：[Python 解释器使用](#)

0.3 应知应会 Checklist

- ☐ 会安装 Python 并检查版本
- ☐ 会使用 python 解释器

- ☐ 会运行 .py 文件
- ☐ 会在终端中运行 Python 程序
- ☐ 会在 IDE 中写并执行程序

0.4 小任务

编写第一个 Python 程序：

```
# hello.py
print("Hello, World!")
```

在终端中运行：

```
python hello.py
```

可以输出 Hello, World! 即表示成功。

Day 1: Python 基础语法

1.1 基本知识点

- 了解 python 的基本语法规则
- 会使用注释与简单输入输出

1.2 学习资料

- 菜鸟教程：
 - [Python3 基础语法](#)
 - [Python3 输入和输出](#)（了解简单的输入输出方式即可）
- Python 官方教程：
 - [Python 速览](#)
 - [输入和输出](#)（了解简单的输入输出方式即可）

1.3 应知应会 Checklist

- ☐ 能在本地运行 Python 程序
- ☐ 掌握 print() 和 input() 用法

1.4 小任务

编写一个程序，输入名字并输出问候语。

```
name = input("请输入你的名字：")
print(f"你好， {name}！ 欢迎学习Python！ ")
```

Day 2: 变量与运算符

2.1 基本知识点

- 变量命名规则
- 了解基本数据类型：整数、浮点数、字符串、布尔值
- 算术运算符： + - * / // % **
- 比较与逻辑运算符： == != > < and or not

2.2 学习资料

- 菜鸟教程：
 - [Python3 基础语法](#)
 - [Python3 基本数据类型](#)（仅需其中 ****数字(Number)*-** 部分）
 - [Python3 数字类型](#)
 - [Python3 运算符](#)

2.3 应知应会 Checklist

- ☐ 变量的定义与赋值
- ☐ 算术运算与逻辑判断
- ☐ 能写出一个简单的计算公式

2.4 小任务

输入两个数字，输出它们的和、差、积、商。

```
a = int(input("请输入数字a: "))
b = int(input("请输入数字b: "))

print("a+b =", a+b)
print("a-b =", a-b)
print("a*b =", a*b)
print("a/b =", a/b)
```

Day 3: 条件语句

3.1 基本知识点

- if elif else 的语法
- 缩进规则与多分支结构

3.2 学习资料

- 菜鸟教程: [Python3 条件控制](#)
- 官方教程: [条件语句](#)

3.3 应知应会 Checklist

- ☐ 能写单分支和多分支结构
- ☐ 理解逻辑运算符在条件语句中的使用
- ☐ 知道 Python 的缩进规则

3.4 小任务

编写一个程序，输入成绩，输出对应等级：

- ≥ 90 : 优秀
- 70~89: 良好
- 60~69: 及格
- < 60 : 不及格

```
score = int(input("请输入成绩: "))
if score >= 90:
    print("优秀")
elif score >= 70:
    print("良好")
elif score >= 60:
    print("及格")
else:
    print("不及格")
```

Day 4: 循环结构

4.1 基本知识点

- while 循环
- for in range() 循环
- break 与 continue

4.2 学习资料

- 菜鸟教程: [Python3 循环语句](#)
- 官方教程: [循环语句](#)

4.3 应知应会 Checklist

- ☐ 理解循环结构
- ☐ 能用 for/while 完成简单循环
- ☐ 理解 break 与 continue

4.4 小任务

打印 1~100 内所有偶数的和。

```
total = 0
for i in range(1, 101):
    if i % 2 == 0:
        total += i
print("1~100 偶数的和:", total)
```

Day 5: 字符串操作

5.1 基本知识点

- 字符串索引与切片
- 常用方法： `len()` , `.upper()` , `.lower()` , `.replace()`
- f-string 格式化

5.2 学习资料

- 菜鸟教程： [Python3 字符串](#)

5.3 应知应会 Checklist

- ☐ 能索引和切片字符串
- ☐ 使用字符串方法进行处理
- ☐ 使用 f-string 格式化输出

5.4 小任务

输入一句话，输出：

1. 字符串长度
2. 全部大写
3. 替换其中的空格为下划线

```
s = input("请输入一句话： ")
print("长度:", len(s))
print("大写:", s.upper())
print("替换:", s.replace(" ", "_"))
```

Day 6: 列表与元组

6.1 基本知识点

- 列表定义与索引
- 列表常用方法： `append()` , `pop()` , `sort()`
- 元组的不可变性

6.2 学习资料

- 菜鸟教程：
 - [Python3 列表](#)
 - [Python3 元组](#)
- 官方教程：
 - 列表：[数据结构](#)
 - 元组：[数据结构](#)

6.3 应知应会 Checklist

- ☐ 定义列表并访问元素
- ☐ 使用列表方法进行修改
- ☐ 理解元组与列表的区别

6.4 小任务

输入 5 个数字存入列表，输出：

1. 最大值和最小值
2. 倒序排列的结果

```
nums = []
for _ in range(5):
    n = int(input("请输入数字: "))
    nums.append(n)

print("最大值:", max(nums))
print("最小值:", min(nums))
print("倒序:", list(reversed(nums)))
```

Day 7: 字典与集合

7.1 基本知识点

- 字典的键值对存取
- 常用方法： `keys()` , `values()` , `items()`
- 集合与集合运算：交集、并集、差集

7.2 学习资料

- 菜鸟教程：
 - [Python3 字典](#)
 - [Python3 集合](#)
- 官方教程：
 - 字典：[数据结构](#)
 - 集合：[数据结构](#)

7.3 应知应会 Checklist

- ☐ 定义并操作字典
- ☐ 遍历字典内容
- ☐ 使用集合运算

7.4 小任务

编写一个程序：

1. 输入一段文本
2. 统计每个单词出现次数（字典）
3. 输出出现次数超过 1 的单词

```
text = input("请输入文本：")
words = text.split()
count = {}
for w in words:
    count[w] = count.get(w, 0) + 1
print("重复出现的单词：")
for k, v in count.items():
    if v > 1:
        print(k, ":", v)
```

Day 8: 函数基础

8.1 基本知识点

- 函数定义与调用
- 参数与返回值

- 默认参数与关键字参数

8.2 学习资料

- 菜鸟教程：[Python3 函数](#)
- 官方教程：[函数](#)

8.3 应知应会 Checklist

- ☐ 能定义并调用函数
- ☐ 理解返回值
- ☐ 会用默认参数

8.4 小任务

编写一个函数 `is_prime(n)`，判断 `n` 是否为质数。

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True

print(is_prime(7)) # True
print(is_prime(10)) # False
```

Day 9: 函数进阶与作用域

9.1 基本知识点

- 局部变量与全局变量
- `return` 多个值（元组）
- 递归函数

9.2 学习资料

- 菜鸟教程：[Python3 作用域](#)
- 官方教程：[函数定义详解](#)

✅ 9.3 应知应会 Checklist

- ☐ 理解局部和全局变量区别
- ☐ 会写递归函数
- ☐ 能用函数分解问题

📄 9.4 小任务

编写一个递归函数，计算斐波那契数列第 n 项。

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)  
  
print(fib(10)) # 55
```

Day 10: 输入输出与文件操作

■ 10.1 基本知识点

- 了解更多输入输出方式
- 文件读写模式： `r` , `w` , `a`
- `open()` 读写文件
- 读取方式： `read()` , `readlines()`
- 写入文件与 `with open`

🔗 10.2 学习资料

- 菜鸟教程：
 - [Python3 输入和输出](#)（了解更多输入输出方式）
 - [Python3 文件 I/O](#)
- 官方教程：[文件和输入输出](#)

✅ 10.3 应知应会 Checklist

- ☐ 对输入输出有更深入理解
- ☐ 会读取文件内容
- ☐ 会写入文件

☐ 使用 `with open` 自动关闭文件

10.4 小任务

编写一个程序，将输入的 3 行文本保存到 `output.txt` 文件中。

```
with open("output.txt", "w", encoding="utf-8") as f:
    for _ in range(3):
        line = input("请输入一行: ")
        f.write(line + "\n")
```

Day 11: 异常处理

11.1 基本知识点

- `try except` 基本语法
- 捕获不同类型异常
- `finally` 子句

11.2 学习资料

- 菜鸟教程：[Python3 异常处理](#)
- 官方教程：[错误和异常](#)

11.3 应知应会 Checklist

- ☐ 知道常见错误类型 (`ValueError`, `ZeroDivisionError`)
- ☐ 会用 `try-except` 捕获异常
- ☐ 理解 `finally` 的作用

11.4 小任务

输入两个数，输出它们的商，若除数为 0，提示错误。

```
try:
    a = int(input("a="))
    b = int(input("b="))
    print("a/b =", a/b)
except ZeroDivisionError:
    print("错误：除数不能为0")
```

Day 12: 初识类与对象

12.1 基本知识点

- 定义类与对象
- `__init__` 构造方法
- 实例属性与方法

12.2 学习资料

- 菜鸟教程：[Python3 类与对象](#)
- 官方教程：[类](#)

12.3 应知应会 Checklist

- ☐ 会定义类
- ☐ 理解 `__init__`
- ☐ 会创建对象并调用方法

12.4 小任务

定义一个 `Student` 类，包含姓名和成绩，提供方法 `show_info()` 输出信息。

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def show_info(self):
        print(f"姓名:{self.name}, 成绩:{self.score}")

s1 = Student("小明", 85)
s1.show_info()
```

Day 13: 类的进阶应用

13.1 基本知识点

- 类方法与实例方法
- 类变量与实例变量
- `__str__` 魔术方法

13.2 学习资料

- 菜鸟教程：[Python3 类与对象](#)
- 官方教程：[类](#)

13.3 应知应会 Checklist

- ☐ 理解类变量与实例变量区别
- ☐ 会用 `__str__` 自定义打印结果
- ☐ 能写简单的类方法

13.4 小任务

编写一个 `Book` 类：

- 属性：书名、作者、价格
- `__str__()` 方法返回 "书名 - 作者 - 价格"

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def __str__(self):
        return f"{self.title} - {self.author} - ¥{self.price}"

b = Book("Python 入门", "张三", 39.9)
print(b)
```

Day 14: 模块与库

14.1 基本知识点

- 什么是模块 (module) 与包 (package)
- 如何导入模块 (import / from ... import ...)
- 常用标准库 (math、random、datetime 等)
- 简单使用第三方库 (以 **pandas** 为例)

14.2 学习资料

- 菜鸟教程：
 - [Python3 模块](#)
 - [pandas 教程](#)
- 官方教程：
 - [模块](#)
 - [标准库简介](#)
 - [标准库简介 —— 第二部分](#)
- pandas 官方文档: [pandas 文档](#)

14.3 应知应会 Checklist

- ☐ 知道如何创建并导入自己的模块
- ☐ 会用 import 导入标准库并调用其中函数
- ☐ 能使用 pandas 读取 CSV 文件并进行简单分析

14.4 小任务

1. **自定义模块**：新建一个 `my_utils.py` 文件，写一个函数 `add(a, b)` 返回两数之和，并在 `task.py` 中导入并调用。
2. **使用标准库**：用 `random` 生成 5 个 1~100 的随机整数。
3. **使用 pandas**：

- 创建一个 `data.csv` 文件（可通过 Excel 或文本编辑器创建该文件），内容如下：

```
name,score
张三,85
李四,90
王五,78
赵六,92
```

- 用 `pandas` 读取该 CSV 文件，并输出平均分和最高分学生的名字。

```
# my_utils.py
def add(a, b):
    return a + b

# task.py
import my_utils
import random
import pandas as pd

# 使用自定义模块
print(my_utils.add(3, 5))

# 使用标准库
nums = [random.randint(1, 100) for _ in range(5)]
print("随机数:", nums)

# 使用 pandas
df = pd.read_csv("data.csv")
print("平均分:", df["score"].mean())
top_student = df.loc[df["score"].idxmax(), "name"]
print("最高分学生:", top_student)
```

大作业

大作业要求

开发一个 **简易学生成绩管理系统**，要求：

1. 可以添加学生（姓名、学号、成绩）
2. 可以查询学生信息
3. 可以保存到文件
4. 使用类来实现

辅助说明

- 使用一个 `Student` 类表示学生，包含**姓名**、**学号**、**成绩**等属性。
- 使用一个 `StudentManager` 类管理学生列表，提供**添加**、**删除**、**查询**、**修改**、**保存至文件**、**加载文件**等方法。
- **查询功能**可以通过学号或姓名进行**查找**，返回对应学生信息。
- **保存功能**可以将学生信息**保存到文件**中。
- **加载功能**可以从文件中读取学生信息并恢复到系统中。并提供三种加载方式：
 - **全量自动加载**：每次启动时加载所有学生信息。
 - **覆盖加载**：每次加载时清空当前数据，重新加载文件中的所有学生信息。
 - **增量加载**：只加载新增或修改的学生信息，避免重复读取。
- **保存的文件格式**可以为：
 - **文本文件**（.txt），每行一个学生信息，格式为 `学号 - 姓名 - 成绩`。例如：


```
202301 - 小明 - 90
202302 - 小红 - 85
```
 - 也可以使用 **CSV**，***JSON**- 等格式，视个人喜好而定。可使用 Python 内置的 `csv` 或 `json` 模块，或使用第三方库如 `pandas` 实现文件读写。
- 仅需实现基本功能，界面可以是**命令行交互式**。
- 代码结构清晰，注释完整，便于理解和维护。
- **【进阶】**可以尝试实现**图形界面**或**Web界面**。

示例功能代码

```
class Student:
    def __init__(self, name, sid, score):
        self.name = name
        self.sid = sid
        self.score = score

    def __str__(self):
        return f"{self.sid} - {self.name} - {self.score}"

class StudentManager:
    def __init__(self):
        self.students = []

    def add_student(self, name, sid, score):
        self.students.append(Student(name, sid, score))

    def show_students(self):
        for s in self.students:
            print(s)

    def save_to_file(self, filename="students.txt"):
        with open(filename, "w", encoding="utf-8") as f:
            for s in self.students:
                f.write(str(s) + "\n")

manager = StudentManager()
manager.add_student("小明", "202301", 90)
manager.add_student("小红", "202302", 85)
manager.show_students()
manager.save_to_file()
```

【选做】进阶大作业：基于 pygame 的简单小游戏

目标：使用 pygame 库开发一个简单的小游戏（如贪吃蛇、打砖块）

简介：

pygame 是一个基于 Python 的游戏开发库，提供了图形绘制、事件处理、音效播放等功能，非常适合初学者练习编程和图形化开发。

学习资料：

- [pygame 官方文档](#)
- [C语言网：Pygame教程（非常详细）](#)
- [Geek Docs：Pygame入门教程](#)

任务目标：

- 使用 pygame 开发一个简单小游戏（如贪吃蛇、打砖块或简易飞机大战）。
- 要求包含以下要素：
 - 游戏窗口与背景绘制
 - 玩家控制（如键盘方向键操作）
 - 简单的碰撞检测（例如蛇撞墙、球撞砖块）
 - 基本的分数统计

基础功能实现 Checklist：

- ☐ 能够使用 pygame 创建游戏窗口并刷新画面
- ☐ 能编写主循环，处理事件（键盘/鼠标）
- ☐ 能绘制简单图形或加载图片
- ☐ 能实现游戏的基本逻辑（例如：角色移动 + 碰撞判定）

小提示：

- 推荐从 **贪吃蛇** 入手，逻辑清晰、代码量适中。
- 如果完成度较高，可以增加功能（计分系统、关卡、音效等）。